FieldOps — Documentação Completa do Projeto

1. VISÃO GERAL

1.1 Nome do produto

FieldOps — Plataforma de Inspeção em Campo

1.2 Resumo executivo

O FieldOps é uma plataforma digital para planejamento, execução, acompanhamento e revisão de inspeções técnicas realizadas em campo.

A solução substitui processos baseados em formulários impressos, planilhas, mensagens e registros informais por um fluxo digital integrado. Técnicos utilizam um aplicativo mobile para consultar inspeções atribuídas, identificar equipamentos, responder checklists, registrar evidências e concluir atividades — inclusive sem conexão com a internet.

Administradores e supervisores utilizam uma interface administrativa web para cadastrar informações, criar modelos de inspeção, agendar atividades, atribuir técnicos, acompanhar execuções, analisar não conformidades e aprovar ou reprovar resultados.

Uma API REST centraliza as regras de negócio, autenticação, autorização, persistência, auditoria e a integração entre o aplicativo mobile e a interface administrativa.

1.3 Declaração da visão

> Para organizações que precisam executar inspeções técnicas de forma padronizada e rastreável, o FieldOps é uma plataforma integrada que permite planejar, executar e revisar inspeções em campo, mesmo sem conexão com a internet. Diferentemente de formulários em papel, planilhas ou aplicativos genéricos, o FieldOps conecta o trabalho do técnico à gestão administrativa, preservando evidências, histórico, localização e regras de negócio em um único fluxo digital.

1.4 Componentes da solução

Componente	Tecnologia	Responsabilidade principal
API REST	Java 17 + Spring Boot 3.5	Segurança, regras de negócio, persistência
Interface Web	React 18 + TypeScript + Vite	Administração e supervisão
Aplicativo Mobile	Expo + React Native 0.86	Execução em campo pelos técnicos
Banco de dados	PostgreSQL (prod) / H2 (dev)	Persistência central
Banco local mobile	SQLite	Operação offline

1.5 Fluxo macro do produto

```
Configuração administrativa
        ↓
Criação do modelo de inspeção
        ↓
Agendamento e atribuição
        ↓
Disponibilização ao técnico
        ↓
Execução em campo
        ↓
Registro de respostas e evidências
```

```
        ↓
Sincronização
        ↓
Revisão do supervisor
        ↓
Aprovação ou solicitação de correção
```

---

2. PROBLEMA E OBJETIVOS

2.1 Problema

Inspeções de equipamentos e instalações frequentemente ocorrem fora do escritório, em locais com conectividade irregular. Em muitos cenários o processo ainda depende de:

- Formulários impressos e planilhas isoladas
- Fotografias sem identificação ou contexto
- Informações enviadas por canais distintos (WhatsApp, e-mail, papel)
- Redigitação manual de dados
- Ausência de fluxo formal de revisão e aprovação

Consequências: retrabalho, perda de dados, decisões baseadas em informações incompletas, dificuldade de auditoria e aumento do risco operacional.

2.2 Objetivos gerais

- Padronizar a execução de inspeções por modelos e checklists versionados
- Permitir execução offline com sincronização posterior
- Associar respostas a evidências, data, usuário, equipamento e localização
- Preservar rastreabilidade completa das operações
- Reduzir o intervalo entre execução em campo e disponibilidade do resultado
- Facilitar identificação e acompanhamento de não conformidades

2.3 Indicadores de sucesso do MVP

O MVP é considerado funcional quando:

- Um supervisor consegue cadastrar dados e criar modelo de inspeção pela interface web
- Uma inspeção pode ser criada e atribuída a um técnico
- O técnico recebe e abre a inspeção no aplicativo mobile
- O checklist é apresentado dinamicamente
- Itens obrigatórios são validados antes da conclusão
- Fotografias podem ser associadas a itens do checklist
- O equipamento pode ser identificado por QR Code
- A localização pode ser registrada quando autorizada
- A inspeção pode ser executada sem conexão após ser baixada
- O reenvio da mesma operação não cria dados duplicados
- O supervisor consegue revisar, aprovar ou reprovar o resultado
- Uma inspeção aprovada fica protegida contra alterações comuns

---

3. PERSONAS

3.1 Carlos — Técnico de campo

Executa inspeções em campo usando o aplicativo mobile. Precisa de interface simples, funcional sem internet, com salvamento automático e indicação clara de progresso e sincronização.

Dores principais: formulários longos, perda de informações, dificuldade de relacionar fotos ao item correto, mensagens de erro técnicas.

3.2 Marina — Supervisora de inspeções

Cria modelos, agenda inspeções, acompanha prazos e revisa resultados pela interface web.

Dores principais: informações em múltiplos canais, atrasos descobertos somente no final, ausência de fluxo formal de aprovação.

3.3 Ana — Administradora do sistema

Cadastra usuários, controla perfis e mantém clientes, locais e equipamentos atualizados.

Dores principais: cadastros espalhados, permissões inconsistentes, dependência da equipe técnica para operações simples.

3.4 Roberto — Responsável do cliente

Consulta resultados, visualiza evidências e acompanha não conformidades (perfil somente leitura, fora do MVP).

---

4. PERFIS DE USUÁRIO E PERMISSÕES

4.1 Perfis implementados na API

Perfil	Enum	Descrição
Administrador	`ADMIN`	Gerencia usuários e cadastros
Supervisor	`SUPERVISOR`	Cria modelos, agenda inspeções, revisa resultados
Técnico	`TECNICO`	Executa inspeções em campo

> **Nota:** O frontend web usa `MANAGER` e `REVIEWER` como roles adicionais — há divergência entre o contrato da API atual e a documentação original (que prevê SUPERVISOR e TECHNICIAN). A API implementada usa `ADMIN`, `SUPERVISOR`, `TECNICO`.

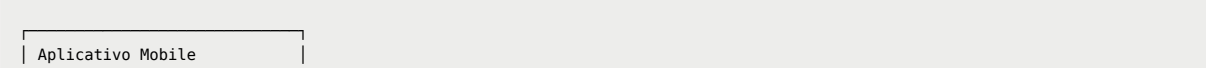
4.2 Princípios de autorização

- Princípio do menor privilégio aplicado em todos os endpoints
- A API valida perfil em toda operação protegida — a interface não é mecanismo suficiente de segurança
- Técnico só pode alterar inspeções atribuídas a ele e em estado compatível
- Registros aprovados ficam protegidos contra edição comum
- Operações não autorizadas retornam erro padronizado sem revelar dados sensíveis

---

5. ARQUITETURA

5.1 Visão geral





5.2 Princípios arquiteturais

- Separação clara entre interface, aplicação, domínio e infraestrutura
- API como ponto central de regras e autorização
- Contrato documentado antes da integração
- Aplicativo mobile preparado para rede instável
- Persistência local como parte da arquitetura, não correção posterior
- Operações de sincronização idempotentes
- Modelos de inspeção versionados e imutáveis após publicação
- Entidades históricas preservadas (soft delete, não exclusão física)
- Erros padronizados e sem exposição de stack trace

5.3 Estrutura de pacotes da API

```
com.experiencefieldops.api
├── config/          # OpenAPI, CORS, Security
├── controllers/     # Endpoints REST
├── dto/             # Contratos de entrada e saída
├── entities/        # Entidades JPA
├── enums/           # Enumerações de domínio
├── exception/       # Tratamento global de erros
├── repositories/    # Interfaces Spring Data JPA
├── security/        # JWT, filtros, UserDetails
├── services/        # Regras de negócio e orquestração
└── util/            # ApiResponse e utilitários
```

---

6. STACK TECNOLÓGICA

6.1 API Backend

Dependência	Versão	Função
Java	17	Linguagem
Spring Boot	3.5.5	Framework principal
Spring Security	(via Boot)	Autenticação e autorização
Spring Data JPA	(via Boot)	ORM e repositórios

Dependência	Versão	Função
JJWT	0.12.6	Geração e validação de JWT
SpringDoc OpenAPI	2.8.9	Swagger UI e documentação
Lombok	(via Boot)	Redução de boilerplate
H2 Database	(via Boot)	Banco em memória para dev
PostgreSQL	(via Boot)	Banco relacional em produção
Spring Boot Validation	(via Boot)	Validação de entrada

6.2 Interface Web (Admin)

Dependência	Versão	Função
React	18.3.1	UI framework
TypeScript	5.6.2	Tipagem estática
Vite	6.0.5	Build tool e dev server
React Router DOM	6.28.0	Roteamento SPA
Axios	1.7.9	HTTP client
Lucide React	0.468.0	Biblioteca de ícones
CSS Modules	—	Estilização escopada

6.3 Aplicativo Mobile

Dependência	Versão	Função
Expo	~57.0.21	Framework base
React Native	0.86.3	Renderização nativa
React	19.2.3	Biblioteca de UI
TypeScript	~6.0.3	Tipagem estática
React Navigation	^7.x	Navegação entre telas
Expo Font	~57.0.4	Fontes customizadas
@expo-google-fonts/inter	^0.4.2	Família de fontes Inter
Lucide React Native	^1.46.0	Ícones
React Native Web	^0.21.2	Suporte à plataforma Web

7. API REST — BACKEND

7.1 Configuração e ambiente

Configuração	Valor
Porta padrão	8080
Banco (dev)	H2 in-memory (`jdbc:h2:mem:fieldopsdb`)
Console H2	`http://localhost:8080/h2-console`
Swagger UI	`http://localhost:8080/swagger-ui.html`
OpenAPI JSON	`http://localhost:8080/v3/api-docs`
Upload máximo por arquivo	10 MB
Upload máximo por request	12 MB
Storage local (dev)	`/uploads`
JWT expiration	86400000 ms (24h)
JWT secret	variável de ambiente `JWT_SECRET`

7.2 Autenticação — JWT

- Estratégia: Bearer Token no header `Authorization`
- Token gerado no login via `/api/auth/login`
- Filtro `JwtAuthenticationFilter` valida token em todas as requisições protegidas
- `CustomUserDetailsService` carrega o usuário do banco por e-mail
- Credenciais e tokens nunca são gravados em logs (RN-007)

Endpoints de autenticação

Método	Endpoint	Acesso	Descrição
POST	`/api/auth/login`	Público	Autentica e retorna JWT
POST	`/api/auth/register`	Público	Registra novo usuário

Exemplo de login

Request:

```
{
  "email": "tecnico@fieldops.local",
  "password": "senha123"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "userId": 1,
  "name": "Carlos Técnico",
  "role": "TECNICO"
}
```

7.3 Padrão de resposta da API

Todos os endpoints protegidos retornam o envelope `ApiResponse<T>`:

```
{
  "success": true,
  "message": "Operação realizada com sucesso",
  "data": { ... }
}
```

Em caso de erro:

```
{
  "timestamp": "2026-10-20T14:30:00Z",
  "status": 422,
  "code": "INSPECTION_REQUIRED_ITEMS_MISSING",
  "message": "A inspeção possui itens obrigatórios sem resposta.",
  "path": "/api/v1/inspections/1/complete",
  "requestId": "req-123"
}
```

7.4 Códigos HTTP utilizados

Código	Situação
200	Operação realizada com sucesso
201	Recurso criado
204	Exclusão sem conteúdo de retorno
400	Dados de entrada inválidos
401	Não autenticado
403	Sem permissão
404	Recurso não encontrado
409	Conflito de estado
422	Violação de regra de negócio
500	Erro interno

7.5 Endpoints — Usuários

Base: `/api/user` — Requer perfil `ADMIN`

Método	Endpoint	Descrição
GET	`/api/user`	Lista todos os usuários ativos
GET	`/api/user/{id}`	Busca usuário por ID
POST	`/api/user`	Cria novo usuário
PUT	`/api/user/{id}`	Atualiza usuário existente
DELETE	`/api/user/{id}`	Soft delete do usuário

UserRequest (POST/PUT)

```
{
  "name": "Carlos Silva",
  "email": "carlos@fieldops.com",
  "passwordHash": "senha_hash",
  "role": "TECNICO",
  "active": true
}
```

UserResponse

```
{
  "id": 1,
  "name": "Carlos Silva",
  "email": "carlos@fieldops.com",
  "role": "TECNICO",
  "active": true,
  "createdAt": "2026-10-01T08:00:00",
  "updatedAt": "2026-10-20T10:00:00"
}
```

7.6 Endpoints — Clientes

Base: `/api/v1/clients`

Método	Endpoint	Descrição
GET	`/api/v1/clients`	Lista clientes
GET	`/api/v1/clients/{id}`	Busca por ID
POST	`/api/v1/clients`	Cria cliente
PUT	`/api/v1/clients/{id}`	Atualiza cliente
PATCH	`/api/v1/clients/{id}/status`	Ativa/inativa
DELETE	`/api/v1/clients/{id}`	Soft delete

7.7 Endpoints — Locais (InspectionSite)

Base: `/api/v1/locations`

Método	Endpoint	Descrição
GET	`/api/v1/locations`	Lista locais
GET	`/api/v1/locations/{id}`	Busca por ID
POST	`/api/v1/locations`	Cria local
PUT	`/api/v1/locations/{id}`	Atualiza local
PATCH	`/api/v1/locations/{id}/status`	Ativa/inativa
DELETE	`/api/v1/locations/{id}`	Soft delete

7.8 Endpoints — Equipamentos

Base: `/api/v1/equipment`

Método	Endpoint	Descrição
GET	`/api/v1/equipment`	Lista equipamentos
GET	`/api/v1/equipment/{id}`	Busca por ID
POST	`/api/v1/equipment`	Cria equipamento
PUT	`/api/v1/equipment/{id}`	Atualiza equipamento
PATCH	`/api/v1/equipment/{id}/status`	Altera status
DELETE	`/api/v1/equipment/{id}`	Soft delete

7.9 Endpoints — Modelos de Inspeção (Templates)

Base: `/api/v1/templates`

Método	Endpoint	Descrição
POST	`/api/v1/templates`	Cria modelo em rascunho (DRAFT)
GET	`/api/v1/templates`	Lista modelos com filtros e paginação
GET	`/api/v1/templates/{id}`	Busca por ID
PUT	`/api/v1/templates/{id}`	Atualiza metadados
PATCH	`/api/v1/templates/{id}/status`	Ativa/desativa modelo
POST	`/api/v1/templates/{id}/versions`	Publica nova versão
GET	`/api/v1/templates/{id}/versions`	Lista versões do modelo
GET	`/api/v1/templates/versions/{versionId}`	Versão completa com seções e itens

Parâmetros de filtro (GET /templates): `title`, `category`, `status`, `page`, `size`

7.10 Endpoints — Seções de Template

Base: `/api/v1/templates/{templateId}/sections`

Método	Endpoint	Descrição
POST	`/api/v1/templates/{id}/sections`	Cria seção
PUT	`/api/v1/templates/{id}/sections/{sectionId}`	Atualiza seção
DELETE	`/api/v1/templates/{id}/sections/{sectionId}`	Remove seção

7.11 Endpoints — Itens de Template

Base: `/api/v1/sections/{sectionId}/items`

Método	Endpoint	Descrição
POST	`/api/v1/sections/{sectionId}/items`	Cria item
PUT	`/api/v1/items/{itemId}`	Atualiza item
DELETE	`/api/v1/items/{itemId}`	Remove item

7.12 Endpoints — Inspeções

Base: `/api/v1/inspections`

Método	Endpoint	Descrição
POST	`/api/v1/inspections`	Cria/agenda inspeção com snapshot automático
GET	`/api/v1/inspections`	Lista com filtros: `technicianId`, `clientId`, `status`
GET	`/api/v1/inspections/{id}`	Busca com snapshot completo dos itens
GET	`/api/v1/inspections/assigned`	Inspeções atribuídas ao técnico
PATCH	`/api/v1/inspections/{id}/start`	Técnico inicia (SCHEDULED → IN_PROGRESS)
PATCH	`/api/v1/inspections/{id}/complete`	Técnico conclui (IN_PROGRESS → COMPLETED)
PATCH	`/api/v1/inspections/{id}/cancel`	Cancela com justificativa obrigatória

Parâmetros adicionais: `requestingUserId` (temporário, será extraído do JWT)

7.13 Endpoints — Respostas

Método	Endpoint	Descrição
GET	`/api/v1/inspections/{id}/responses`	Lista respostas da inspeção
POST	`/api/v1/inspections/{id}/responses`	Cria ou atualiza resposta

7.14 Endpoints — Evidências

Método	Endpoint	Descrição
POST	`/api/v1/inspections/{id}/evidences`	Upload de evidência (multipart/form-data)
GET	`/api/v1/inspections/{id}/evidences`	Lista evidências da inspeção
GET	`/api/v1/evidences/{id}`	Busca metadados de evidência
DELETE	`/api/v1/evidences/{id}`	Soft delete auditado

Parâmetros do upload: `file` (obrigatório), `responseId` (opcional), `nonConformityId` (opcional), `uploadedByUserId`

7.15 Endpoints — Não Conformidades

Método	Endpoint	Descrição
POST	`/api/v1/inspections/{id}/non-conformities`	Registra NC vinculada à inspeção
GET	`/api/v1/inspections/{id}/non-conformities`	Lista NCs da inspeção
GET	`/api/v1/non-conformities/{id}`	Busca NC por ID
PUT	`/api/v1/non-conformities/{id}`	Atualiza dados da NC

Método	Endpoint	Descrição
PATCH	`/api/v1/non-conformities/{id}/status`	Altera status (DISMISSED exige justificativa)

7.16 Endpoints — Revisão de Inspeção

Método	Endpoint	Descrição
POST	`/api/v1/inspections/{id}/reviews`	Supervisor inicia revisão
GET	`/api/v1/inspections/{id}/reviews`	Histórico de revisões
GET	`/api/v1/reviews/{id}`	Busca revisão por ID
POST	`/api/v1/reviews/{id}/approve`	Aprova inspeção
POST	`/api/v1/reviews/{id}/reject`	Reprova com motivo obrigatório

7.17 Endpoints — Sincronização

Método	Endpoint	Descrição
POST	`/api/v1/sync/batch`	Processa lote de operações da outbox
GET	`/api/v1/inspections/{id}/sync-status`	Status de sincronização da inspeção

Lote sempre retorna 200 — o status de cada operação individual está no corpo da resposta:

```
{
  "succeeded": 3,
  "failed": 1,
  "skipped": 0,
  "results": [
    { "operationId": "uuid", "status": "APPLIED", "entityVersion": 2 },
    { "operationId": "uuid", "status": "ALREADY_APPLIED" }
  ]
}
```

Status por operação: `APPLIED`, `ALREADY_APPLIED`, `REJECTED`, `CONFLICT`, `DEPENDENCY_FAILED`

7.18 Endpoints — Audit Log

Método	Endpoint	Descrição
GET	`/api/v1/audit-logs`	Lista eventos de auditoria
GET	`/api/v1/audit-logs/{inspectionId}`	Histórico de uma inspeção

8. MODELO DE DADOS

8.1 Diagrama de relacionamentos

```
USER
 ├── cria/agenda ── N INSPECTION
 ├── executa ──────── N INSPECTION
 ├── revisa ──────── N INSPECTION_REVIEW
 └── realiza ─────── N AUDIT_LOG

CLIENT 1 ── N INSPECTION_SITE
INSPECTION_SITE 1 ── 0..N EQUIPMENT

INSPECTION_TEMPLATE 1 ── N INSPECTION_TEMPLATE_VERSION
INSPECTION_TEMPLATE_VERSION 1 ── N TEMPLATE_SECTION
TEMPLATE_SECTION 1 ── N TEMPLATE_ITEM

INSPECTION_TEMPLATE_VERSION 1 ── N INSPECTION
```

```
CLIENT 1 ── N INSPECTION
INSPECTION_SITE 1 ── N INSPECTION
EQUIPMENT 0..1 ── N INSPECTION
USER 1 ── N INSPECTION

INSPECTION 1 ── N INSPECTION_ITEM_SNAPSHOT
INSPECTION_ITEM_SNAPSHOT 1 ── 0..1 INSPECTION_RESPONSE
INSPECTION_RESPONSE 1 ── 0..N EVIDENCE

INSPECTION 1 ── 0..N NON_CONFORMITY
NON_CONFORMITY 1 ── 0..N EVIDENCE

INSPECTION 1 ── 0..N INSPECTION_REVIEW
INSPECTION 1 ── N AUDIT_LOG
```

8.2 Entidades principais

User (tabela: `users`)

Campo	Tipo	Observação
id	Long	PK, auto-increment
name	String(150)	Não nulo
email	String(150)	Único, não nulo
passwordHash	String	Não nulo
role	UserRole	Enum: ADMIN, SUPERVISOR, TECNICO
active	Boolean	Não nulo
version	Long	Controle otimista
createdAt	LocalDateTime	Auditoria automática
updatedAt	LocalDateTime	Auditoria automática
deletedAt	LocalDateTime	Soft delete

Client (tabela: `clients`)

Campo	Tipo	Observação
id	Long	PK
name	String(150)	Não nulo
document	String(20)	CNPJ/CPF
email	String(150)	Contato
phone	String(30)	Contato
status	ClientStatus	ACTIVE / INACTIVE
version	Long	Controle otimista
createdAt	LocalDateTime	—
updatedAt	LocalDateTime	—
deletedAt	LocalDateTime	Soft delete

Equipment (tabela: `equipments`)

Campo	Tipo	Observação
id	Long	PK
name	String(150)	Não nulo
description	String(255)	—
qrCode	String(36)	Único, não nulo
status	EquipmentStatus	ACTIVE / MAINTENANCE / INACTIVE
site	InspectionSite	FK, não nulo

Campo	Tipo	Observação
version	Long	—
createdAt	LocalDateTime	—
updatedAt	LocalDateTime	—
deletedAt	LocalDateTime	Soft delete

Inspection (tabela: `inspections`)

Campo	Tipo	Observação
id	Long	PK
client	Client	FK, não nulo
site	InspectionSite	FK, não nulo
assignedTo	User	Técnico responsável, FK
templateVersion	InspectionTemplateVersion	FK, não nulo
equipment	Equipment	FK, opcional
scheduledDate	LocalDate	Data prevista
status	InspectionStatus	Ver estados abaixo
startedAtDevice	LocalDateTime	Hora do dispositivo
startedAtServer	LocalDateTime	Hora de recebimento
completedAtDevice	LocalDateTime	—
completedAtServer	LocalDateTime	—
cancelledBy	User	FK, opcional
cancelledAt	LocalDateTime	—
cancellationReason	String(500)	Obrigatório ao cancelar
version	Long	Controle otimista

InspectionTemplate (tabela: `inspection_templates`)

Campo	Tipo	Observação
id	Long	PK
title	String(150)	Não nulo
description	String(500)	—
category	String(100)	Não nulo
status	TemplateStatus	DRAFT / PUBLISHED / INACTIVE
currentVersion	Integer	Última versão publicada (0 = nunca)

NonConformity (tabela: `non_conformities`)

Campo	Tipo	Observação
id	Long	PK
inspection	Inspection	FK, não nulo
snapshotItem	InspectionItemSnapshot	FK, opcional
title	String(255)	Não nulo
description	TEXT	Obrigatório para CRITICAL
severity	Severity	LOW / MEDIUM / HIGH / CRITICAL
status	NonConformityStatus	OPEN / IN_PROGRESS / RESOLVED / DISMISSED
dismissalReason	String(500)	Obrigatório para DISMISSED
evidenceAttached	Boolean	Flag informativa

8.3 Enumerações

InspectionStatus

```
SCHEDULED     — Agendada e atribuída ao técnico
IN_PROGRESS   — Técnico iniciou
COMPLETED     — Técnico concluiu
UNDER_REVIEW  — Em revisão pelo supervisor
APPROVED      — Aprovada
REJECTED      — Reprovada, aguarda correção
CANCELLED     — Cancelada com justificativa
```

UserRole

```
ADMIN
SUPERVISOR
TECNICO
```

ResponseType (tipos de resposta do MVP)

```
TEXT_SHORT
TEXT_LONG
NUMBER
BOOLEAN
CONFORMITY
SINGLE_CHOICE
DATE
```

Severity (criticidade de NC)

```
LOW
MEDIUM
HIGH
CRITICAL
```

EquipmentStatus

```
ACTIVE
MAINTENANCE
INACTIVE
```

8.4 Snapshot e versionamento

Ao criar uma inspeção, a API automaticamente:

1. Valida cliente, local, equipamento, técnico e versão do modelo
2. Cria a inspeção
3. Copia todas as seções e itens da versão publicada
4. Gera registros `InspectionItemSnapshot` para cada item
5. Registra evento de auditoria

O snapshot garante que o checklist respondido pelo técnico nunca seja afetado por alterações futuras no modelo.

---

9. REGRAS DE NEGÓCIO PRINCIPAIS

9.1 Usuários e acesso

- **RN-001:** Somente usuários ativos podem iniciar sessão
- **RN-002:** Inativação não exclui registros históricos
- **RN-003:** A API valida perfil em todas as operações protegidas
- **RN-007:** Credenciais e tokens nunca são gravados em logs

9.2 Clientes, locais e equipamentos

- **RN-009:** Todo local pertence a um cliente
- **RN-010:** Todo equipamento pertence a um local no MVP
- **RN-011:** O QR Code de um equipamento deve ser único
- **RN-012:** Equipamento inativo não pode ser usado em nova inspeção
- **RN-013:** Registros em uso devem ser inativados, não excluídos fisicamente

9.3 Modelos de inspeção

- **RN-015:** Modelo precisa de título, categoria e ao menos uma seção com item para publicação
- **RN-019:** Versão publicada não pode ser alterada de forma destrutiva
- **RN-020:** Alterações em modelo publicado geram nova versão
- **RN-021:** Inspeção preserva snapshot dos itens da versão utilizada

9.4 Planejamento

- **RN-024:** Inspeção vinculada a versão publicada
- **RN-025:** Inspeção requer cliente, local, técnico e data prevista
- **RN-027:** Apenas técnico ativo pode receber nova atribuição
- **RN-029:** Cancelamento registra usuário, data e justificativa
- **RN-030:** Inspeção aprovada não pode ser cancelada pelo fluxo comum

9.5 Execução

- **RN-032:** Apenas inspeções atribuídas (SCHEDULED) podem ser iniciadas
- **RN-033:** Apenas o técnico responsável pode responder a inspeção
- **RN-037:** Itens obrigatórios devem ser respondidos antes da conclusão
- **RN-039:** Item crítico não conforme exige ao menos uma evidência

9.6 Sincronização

- **RN-067:** Cada operação enviada possui identificador idempotente único
- **RN-068:** Reenvio da mesma operação não cria duplicidade
- **RN-069:** Operações processadas respeitando dependências
- **RN-073:** O servidor é a fonte oficial após confirmação da sincronização

9.7 Revisão e aprovação

- **RN-079:** Apenas supervisor autorizado pode iniciar revisão
- **RN-080:** Reprovação exige motivo obrigatório
- **RN-082:** Inspeção aprovada fica bloqueada para edição comum

- **RN-083:** Correções após reprovação preservam histórico anterior

9.8 Auditoria

- **RN-086:** Alterações críticas registram usuário, data, ação e entidade
- **RN-088:** Entidades históricas não podem ser excluídas fisicamente pelo fluxo comum
- **RN-089:** A API rejeita transições de estado inválidas

---

10. INTERFACE ADMINISTRATIVA WEB

10.1 Stack e configuração

Item	Detalhe
Dev server	http://localhost:5173
Backend esperado	http://localhost:8080
Proxy Vite	`/api/*` → `http://localhost:8080/api/*`
Build	`tsc -b && vite build`
Linter	ESLint + oxlint

10.2 Autenticação

- Token JWT armazenado em `localStorage` (`fieldops_token`, `fieldops_user`)
- Interceptor Axios global: injeta `Authorization: Bearer <token>` em toda requisição
- Erro 401: remove token e redireciona para `/login`
- `AuthContext` gerencia o estado de autenticação globalmente
- Rotas protegidas: redirecionam para `/login` se não autenticadas
- Rotas públicas: redirecionam para `/dashboard` se já autenticadas

10.3 Estrutura de pastas

```
src/
├── assets/          # Imagens e SVGs estáticos
├── components/
│   ├── Layout       # Wrapper sidebar + conteúdo
│   ├── Sidebar      # Navegação lateral
│   ├── Modal        # Dialog acessível (ESC / clique fora)
│   └── Pagination   # Paginação inteligente
├── contexts/
│   └── AuthContext  # Estado global de autenticação
├── hooks/
│   └── usePagination # Hook genérico (10 itens/página)
├── pages/
│   ├── Login        # Tela de login
│   ├── Dashboard    # Visão geral
│   ├── Usuarios     # CRUD usuários
│   ├── Clientes     # CRUD clientes
│   ├── Locais       # CRUD locais
│   ├── Equipamentos # CRUD equipamentos
│   └── NotFound     # 404
├── services/
│   ├── api.ts       # Instância Axios + interceptors
│   ├── authService  # Login, logout, sessão
│   └── userService  # CRUD usuários
└── types/
    └── index.ts     # Interfaces e tipos globais
```

10.4 Páginas implementadas

Login (`/login`)

- Painel esquerdo: branding FieldOps
- Painel direito: formulário com e-mail e senha
- Toggle de visibilidade da senha
- "Lembrar-me" (visual apenas, sem persistência diferenciada)
- Tratamento de erro inline
- Após login, redireciona para `/dashboard`

Dashboard (`/dashboard`)

- 4 stat cards clicáveis: Usuários, Clientes, Locais, Equipamentos
- Barra de status rápido com indicadores coloridos
- Tabela de usuários paginada (10/página) com atalho de edição
- Feed de atividade recente (6 últimos usuários atualizados)
- Gráfico de barras por perfil (ADMIN, MANAGER, INSPECTOR, REVIEWER)
- **Locais e Equipamentos** nos cards usam valores fixos (39 e 58) — não buscam do backend

Usuários (`/usuarios`)

- Listagem com busca por nome/e-mail e filtros por perfil e status
- Modal de criação: nome*, e-mail*, senha*, perfil, ativo
- Modal de edição: mesmo formulário sem campo senha
- Exclusão com `confirm()` nativo
- Roles: ADMIN, MANAGER, INSPECTOR, REVIEWER

Clientes (`/clientes`)

- Busca por nome, e-mail ou CNPJ
- Modal: nome*, e-mail de contato, telefone, CNPJ/CPF, ativo

Locais (`/locais`)

- Vinculados a clientes (select populado via `/api/v1/clients`)
- Busca por nome, cliente ou cidade
- Modal: nome*, endereço, cidade, UF (2 chars), cliente, ativo

Equipamentos (`/equipamentos`)

- Vinculados a locais
- Filtro por status além da busca textual
- Status: ATIVO, EM_MANUTENCAO, INATIVO
- Categorias: Elétrico, Mecânico, HVAC, Hidráulico, Instrumentação, Segurança, Estrutural
- Modal: nome*, nº série, categoria, local (select), status

10.5 Mapeamento de endpoints (frontend × API)

Método	Endpoint	Usado em
POST	`/api/auth/login`	Login
GET	`/api/user`	Dashboard, Usuários

Método	Endpoint	Usado em
POST	`/api/user`	Criar usuário
PUT	`/api/user/:id`	Editar usuário
DELETE	`/api/user/:id`	Excluir usuário
GET	`/api/v1/clients`	Dashboard, Clientes, Locais
POST	`/api/v1/clients`	Criar cliente
PUT	`/api/v1/clients/:id`	Editar cliente
DELETE	`/api/v1/clients/:id`	Excluir cliente
GET	`/api/v1/locations`	Locais, Equipamentos
POST	`/api/v1/locations`	Criar local
PUT	`/api/v1/locations/:id`	Editar local
DELETE	`/api/v1/locations/:id`	Excluir local
GET	`/api/v1/equipment`	Equipamentos
POST	`/api/v1/equipment`	Criar equipamento
PUT	`/api/v1/equipment/:id`	Editar equipamento
DELETE	`/api/v1/equipment/:id`	Excluir equipamento

> **Inconsistência conhecida:** usuários usam `/api/user` (sem versionamento), demais entidades usam `/api/v1/...`

10.6 Tipagem global (src/types/index.ts)

```ts
type UserRole = 'ADMIN' | 'MANAGER' | 'INSPECTOR' | 'REVIEWER'

interface User {
  id: number; name: string; email: string;
  role: UserRole; active: boolean;
  createdAt: string; updatedAt: string;
}

interface Client {
  id: number; name: string; contactEmail: string;
  phone?: string; document?: string; active: boolean;
  createdAt: string; updatedAt: string;
}

interface Location {
  id: number; name: string; address?: string;
  city?: string; state?: string; clientId: number;
  clientName?: string; active: boolean;
}

interface Equipment {
  id: number; name: string; serialNumber?: string;
  qrCode?: string; category?: string;
  status: string; locationId: number;
  locationName?: string; clientName?: string;
}
```

11. APLICATIVO MOBILE

11.1 Stack e plataformas suportadas

Item	Detalhe
Framework	Expo ~57 + React Native 0.86
Linguagem	TypeScript ~6.0
Navegação	React Navigation v7
Fontes	Inter (400, 500, 600, 700) via @expo-google-fonts
Plataforma prioritária	Android
Também suportado	iOS e Web (react-native-web)

11.2 Estrutura de arquivos

```
fieldops/
├── App.tsx                  # Raiz — carrega fontes e monta navigator
├── index.ts                 # Entry point — registra via Expo
├── app.json                 # Config Expo (ícone, orientação, plugins)
├── assets/                  # Ícones e imagens
├── navigation/
│   └── AppNavigator.tsx     # Stack + Bottom Tabs
├── screens/
│   ├── LoginScreen.tsx      # Tela de autenticação (implementada)
│   ├── DashboardScreen.tsx  # Em desenvolvimento
│   ├── InspectionsScreen.tsx # Em desenvolvimento
│   ├── EquipmentScreen.tsx  # Em desenvolvimento
│   └── ProfileScreen.tsx    # Em desenvolvimento
└── components/
    └── AppText.tsx          # Texto com fonte Inter padrão
```

11.3 Arquitetura de navegação

```
Stack Navigator (raiz)
├── Login (headerShown: false)
└── Main (headerShown: false)
    └── Bottom Tab Navigator
        ├── Dashboard
        ├── Inspeções
        ├── Equipamentos
        └── Perfil
```

11.4 Telas

LoginScreen (implementada)

- Logo do app
- Campo de e-mail
- Campo de senha com toggle de visibilidade (Eye/EyeOff)
- Toggle "Lembrar-me" (visual, sem estado persistido)
- Botão "Logar" → navega para `Main`
- Link "Esqueci minha senha" (sem ação implementada)
- **Sem integração com backend** — autenticação é apenas `navigation.navigate('Main')`

Dashboard, Inspeções, Equipamentos, Perfil

- Telas em desenvolvimento — retornam texto placeholder

11.5 Componente AppText

Wrapper sobre o `Text` do React Native que injeta automaticamente a fonte `Inter_400Regular`. Garante consistência tipográfica em todo o app.

11.6 Fontes carregadas

Variante	Uso
Inter_400Regular	Corpo de texto padrão
Inter_500Medium	Textos intermediários
Inter_600SemiBold	Links e destaques
Inter_700Bold	Botões e títulos

O app não renderiza nada enquanto as fontes não estão carregadas (`if (!fontsLoaded) return null`).

11.7 Scripts disponíveis

```
npm start            # Inicia o Expo Dev Server
npm run android      # Abre no emulador/dispositivo Android
npm run ios          # Abre no simulador iOS
npm run web          # Abre no navegador
```

11.8 Arquitetura sugerida para implementação completa

```
src/
├── app/                    # Rotas do Expo Router
├── features/
│   ├── auth/
│   ├── inspections/
│   ├── checklist/
│   ├── evidence/
│   ├── scanner/            # QR Code
│   ├── location/
│   └── synchronization/
├── components/             # Componentes compartilhados
├── design-system/          # Tokens e tema
├── application/            # Casos de uso
├── domain/                 # Tipos e regras independentes
├── infrastructure/
│   ├── api/
│   ├── database/           # SQLite
│   ├── repositories/
│   └── sync/               # Outbox
├── hooks/
└── utils/
```

---

12. OFFLINE E SINCRONIZAÇÃO

12.1 Estratégia offline-first

O aplicativo mobile armazena localmente todos os dados necessários para execução offline:

- Inspeções atribuídas e snapshot dos itens
- Respostas e referências de evidências
- Não conformidades
- Revisões e solicitações de correção pertinentes

12.2 Fluxo de salvamento local

```
Técnico responde um item
        ↓
App valida o valor
        ↓
Resposta salva no SQLite
        ↓
Operação criada na outbox
        ↓
Interface exibe "Salvo no dispositivo"
```

A interface não aguarda resposta da API para considerar o dado salvo localmente.

12.3 Outbox de operações

Cada alteração sincronizável gera uma operação com:

Campo	Descrição
operationId	UUID idempotente único
entityType	Tipo da entidade (ex: INSPECTION_RESPONSE)
entityId	ID global
operationType	UPSERT, DELETE
payload	Dados da operação
dependencies	IDs de operações que devem ser processadas antes
attempts	Número de tentativas
status	PENDING, SENT, FAILED

12.4 Envio do lote

```
Conectividade disponível
        ↓
App seleciona operações pendentes
        ↓
Operações ordenadas por dependência
        ↓
Lote enviado com IDs idempotentes
        ↓
API processa cada operação individualmente
        ↓
App registra resultado por operação
        ↓
Confirmadas: saem da fila
Falhas: permanecem pendentes
```

12.5 Estrutura do lote (SyncBatchRequest)

```json
{
  "deviceId": "uuid-do-dispositivo",
  "operations": [
    {
      "operationId": "uuid-idempotente",
      "entityType": "INSPECTION_RESPONSE",
      "entityId": "uuid-resposta",
      "operationType": "UPSERT",
      "baseVersion": 0,
      "payload": {
        "inspectionItemId": "uuid-item",
        "conformity": "NON_CONFORMING",
        "observation": "Cabo elétrico danificado."
```

```
        }
      }
    ]
  }
```

12.6 Regras críticas de sincronização

- **RN-067/068:** Operação com mesmo `operationId` não cria duplicidade
- **RN-069:** Operações processadas em ordem de dependência
- **RN-070:** Falha em uma operação não cancela o restante do lote
- **RN-073:** Servidor é a fonte oficial após confirmação
- **RN-076:** Dados aprovados no servidor não aceitam atualização tardia comum

---

13. FLUXOS DE NEGÓCIO

13.1 Fluxo completo de uma inspeção

```
1. ADMINISTRADOR cadastra usuários, clientes, locais, equipamentos
2. SUPERVISOR cria modelo de inspeção em rascunho
3. SUPERVISOR adiciona seções e itens ao modelo
4. SUPERVISOR publica versão do modelo
5. SUPERVISOR agenda inspeção e atribui técnico
6. API gera snapshot dos itens automaticamente
7. TÉCNICO sincroniza — recebe inspeção no app
8. TÉCNICO abre detalhes e confirma equipamento (QR Code)
9. TÉCNICO inicia a inspeção → status: IN_PROGRESS
10. TÉCNICO responde checklist item a item (salvo localmente)
11. TÉCNICO registra fotografias, observações e NCs
12. TÉCNICO conclui → status local: aguardando sincronização
13. App envia outbox → API processa → status: COMPLETED
14. SUPERVISOR revisa respostas, evidências e NCs
15. SUPERVISOR aprova ou reprova
    - APROVADO → status: APPROVED, bloqueado para edição
    - REPROVADO → status: REJECTED, técnico recebe para correção
```

13.2 Transições de estado da inspeção

De	Para	Ação
—	SCHEDULED	Agendamento pelo supervisor
SCHEDULED	IN_PROGRESS	Técnico inicia
IN_PROGRESS	COMPLETED	Técnico conclui
COMPLETED	UNDER_REVIEW	Supervisor inicia revisão
UNDER_REVIEW	APPROVED	Supervisor aprova
UNDER_REVIEW	REJECTED	Supervisor reprova
REJECTED	IN_PROGRESS	Técnico corrige e reenvia
Qualquer[1]	CANCELLED	Cancelamento com justificativa

> [1] Exceto APPROVED

13.3 Tratamento de exceções relevantes

Situação	Comportamento
App fechado durante preenchimento	Respostas confirmadas permanecem no SQLite
Falha ao enviar foto	Dados textuais preservados, foto permanece pendente

Situação	Comportamento
Inspeção cancelada offline	Na sincronização, preserva alterações locais e informa cancelamento
Sessão expirada	Tenta renovar; se falhar, mantém dados locais e solicita reautenticação
Operação reenviada	API reconhece `operationId` e impede duplicidade
Modelo alterado após agendamento	Inspeção continua usando snapshot da versão original

14. CASOS DE USO PRINCIPAIS

UC-01 — Autenticar usuário

Ator: qualquer usuário ativo

Fluxo: usuário informa credenciais → API valida → retorna JWT → sessão criada

UC-04 — Criar modelo de inspeção

Ator: supervisor

Fluxo: cria rascunho → adiciona seções e itens → define tipos de resposta → publica versão imutável

UC-06 — Agendar e atribuir inspeção

Ator: supervisor

Fluxo: seleciona modelo publicado, cliente, local, técnico e data → API cria inspeção e snapshot

UC-08 — Identificar equipamento por QR Code

Ator: técnico

Fluxo: app solicita permissão de câmera → lê QR Code → busca equipamento localmente ou na API → técnico confirma vínculo

UC-09 — Iniciar inspeção

Ator: técnico

Fluxo: confirma dados → app registra hora do dispositivo → altera estado para IN_PROGRESS → exibe checklist

UC-10 — Responder checklist

Ator: técnico

Fluxo: app renderiza componente por tipo → técnico informa resposta → validação → salvo no SQLite → outbox atualizada

UC-12 — Registrar não conformidade

Ator: técnico

Fluxo: informa título, descrição e criticidade → adiciona evidências → salvo localmente → adicionado à outbox

UC-14 — Sincronizar alterações

Ator: sistema

Fluxo: identifica operações pendentes → ordena por dependência → envia lote com IDs idempotentes → API processa → app atualiza estado

UC-16 — Revisar inspeção

Ator: supervisor

Fluxo: abre inspeção → navega por seções → analisa respostas, evidências e NCs → decide aprovar ou reprovar

UC-17 — Aprovar ou reprovar inspeção

Ator: supervisor

- **Aprovação:** registra supervisor, data e comentário → status APPROVED → conteúdo protegido
- **Reprovação:** motivo obrigatório → status REJECTED → técnico recebe para correção

---

15. SEGURANÇA

15.1 Implementação na API

Aspecto	Implementação
Autenticação	JWT Bearer Token (JJWT 0.12.6)
Expiração do token	24 horas (configurável via `JWT_SECRET`)
Hash de senha	Spring Security `PasswordEncoder`
Autorização	`@PreAuthorize` por endpoint
Filtro JWT	`JwtAuthenticationFilter` em toda requisição
CORS	Configurado em `SecurityConfig`
Logs	Tokens e senhas nunca registrados
Erros externos	Stack trace não exposto ao cliente

15.2 Segurança no frontend web

- Token em `localStorage` — removido automaticamente em logout ou erro 401
- Interceptor Axios injeta token em toda chamada
- Rotas protegidas redirecionam para login se não autenticadas

15.3 Segurança no mobile

- Tokens armazenados em mecanismo seguro do dispositivo (Expo SecureStore — previsto)
- Sessão local preservada para operação offline
- Permissões de câmera e localização solicitadas com consentimento explícito

---

16. PONTOS DE ATENÇÃO E PENDÊNCIAS

16.1 API Backend

Item	Situação
`requestingUserId` passado como query param	Temporário — deve vir do JWT quando auth estiver integrado
`technicianId` passado como query param em `/assigned`	Idem
Roles no `UserRole` enum: `ADMIN, SUPERVISOR, TECNICO`	Diverge da documentação original (TECHNICIAN)
Roles no frontend web: `ADMIN, MANAGER, INSPECTOR, REVIEWER`	Diverge da API — não sincronizados
Endpoint de usuários sem versionamento: `/api/user`	Inconsistente com demais endpoints `/api/v1/...`

16.2 Interface Web

Item	Situação
Locais e Equipamentos nos stat cards do Dashboard	Valores fixos (39 e 58), não buscam do backend
`react-hook-form` instalado	Não utilizado — formulários são controlados manualmente
"Lembrar-me" no login	Visual apenas, sem comportamento diferenciado
`StatCard` e `Logo`	Componentes declarados mas não integrados nas páginas
Sem testes automatizados	—

16.3 Aplicativo Mobile

Item	Situação
Login	Sem integração com backend (apenas navega para Main)
Dashboard, Inspeções, Equipamentos, Perfil	Stubs — aguardam implementação
SQLite	Não implementado ainda
Sincronização offline	Não implementada ainda
QR Code	Não implementado ainda
Localização	Não implementada ainda
"Lembrar-me"	Visual apenas, sem estado persistido

---

17. INSTRUÇÕES PARA EXECUTAR O PROJETO

17.1 API Backend

Pré-requisitos: Java 17+, Maven 3.8+

```
# Na pasta api/
./mvnw spring-boot:run

# Ou no Windows:
mvnw.cmd spring-boot:run
```

- API disponível em: `http://localhost:8080`
- Swagger UI: `http://localhost:8080/swagger-ui.html`
- H2 Console: `http://localhost:8080/h2-console`
- JDBC URL para H2: `jdbc:h2:mem:fieldopsdb`

Variáveis de ambiente (produção):

```
JWT_SECRET=<chave-base64-minimo-256-bits>
```

17.2 Interface Web

Pré-requisitos: Node.js 18+

```
# Instalar dependências
npm install

# Iniciar dev server
npm run dev
```

- Interface disponível em: `http://localhost:5173`

- Proxy `/api/*` → `http://localhost:8080/api/*`

17.3 Aplicativo Mobile

Pré-requisitos: Node.js 18+, Expo CLI, Android Studio ou Xcode

```
# Instalar dependências
npm install

# Iniciar Expo Dev Server
npm start

# Abrir em plataforma específica
npm run android
npm run ios
npm run web
```

18. GLOSSÁRIO

Termo	Definição
Checklist	Lista de itens de verificação pertencentes a uma versão publicada de um modelo de inspeção
Snapshot	Cópia imutável dos itens do checklist gerada no momento do agendamento de uma inspeção
Outbox	Fila local de operações pendentes de sincronização armazenada no SQLite do dispositivo
Não Conformidade (NC)	Desvio ou irregularidade identificada durante a execução da inspeção
Versão publicada	Estado imutável de um modelo de inspeção que pode ser usado em novos agendamentos
Idempotência	Propriedade que garante que o reenvio de uma mesma operação não cria dados duplicados
Soft delete	Marcação lógica de exclusão via campo `deletedAt`, sem remoção física do banco
Controle otimista	Mecanismo que usa o campo `version` para detectar edições concorrentes
InspectionSite	Local ou instalação pertencente a um cliente onde inspeções são realizadas
Evidence	Arquivo (principalmente foto) associado a uma resposta ou não conformidade